```
35
-   describe "get_contact!/1" do
-     test "no contact raises an error" do
-       assert_raise(Ecto.NoResultsError, fn ->
-         ContactStore.get_contact!(0)
40       end)
-     end
-
-     test "a contact is returned" do
-       assert {:ok, contact} = ContactStore.upsert_contact(@incoming)
45       assert ContactStore.get_contact!(contact.id) == contact
-     end
-   end
- end
```

The use statement on line 2 has changed to PhoneApp.DataCase. This module was created by the Phoenix generator—you can find the module at test/support/data_case.ex. PhoneApp.DataCase is based on ExUnit.Case, but it sets up the PhoneApp.Repo so it can be used in our test module.

These tests largely speak for themselves, but the module attribute on line 7 is new. Test modules are just normal Elixir code! This means we can use module attributes to simplify our testing code. We don't do it here, but we can even define helper functions in our test to extract repeated code.

We use a mix of pattern assertions (line 16) and strict equality assertions (line 18) throughout our test module. However, line 38 introduces the assert_raise/2 assertion. This assertion macro accepts an anonymous function and asserts that a particular error is raised from it.

To demonstrate the importance of tests, these tests revealed a bug in the original Repo.insert/2 function. The fix was to add returning: true to the insert options. If you run into an error with that particular test, make sure that option is used in your Repo.insert/2 call in ContactStore.upsert_contact/1.

Let's test our outbound HTTP requests next.

Test External API Requests

There are many opinions on how to test external requests. Rubyists use mocking (RSpec and Minitest both include mocking capabilities) at the method level or mocking at the HTTP level (VCR) to test external dependencies. Personally, I always reached for VCR because of how simple and powerful it was.

The solution for testing external requests in Elixir is largely the same: mock the function or mock the HTTP request. Personally, I usually mock at the